Getting Started with DevOps AI Skills

Teach your agent your runbooks, using Agent Skills, APM, and Pulumi

Engin Diri · Adam Gordon Bell · Pulumi

Engin Diri

Senior Solutions Architect at Pulumi

@_ediri engin-diri dirien

Building platform tooling and infrastructure-as-code. Helping teams ship cloud infrastructure faster. With and without agents.

Adam Gordon Bell

Community Engineer at Pulumi

@adamgordonbell adamgordonbell adamgordonbell

Host of the CoRecursive podcast.

Telling the stories behind the code.

Housekeeping & Agenda

Housekeeping

- We'll **present and demo**, so you don't need to code along live
- Ask questions **any time**; treat this as a conversation
- Slides, chapters, and the finished skills go home in the **repo** (QR at the end)
- Everything I show, you can run yourself later. One chapter at a time.

Today's Agenda

- **What** — an Agent Skill, up close
- **Why** — DevOps work is skill-shaped
- **Get** — find & evaluate existing skills
- **Wire** — one manifest for the team: skills, MCP, LSP, hooks
- **Use** — same prompt, configured vs not — live
- **Build** — your own skills, done well

What

an Agent Skill, up close

what · why · get · wire · use · build

Your agent is smart. And generic.

It writes great code. It does **not** know your `golden paths`, your `runbooks`, or your `tagging and region conventions.`

So you paste the same context into every prompt. Again. And again. And it still reaches for the wrong component or forgets to `pulumi preview` first.

A **skill** packages that context **once**, and the agent loads it `automatically` when the task calls for it.

A skill is a folder with a `SKILL.md`

```
---
name: pulumi-stack-bootstrap
description: >-
  Scaffold a new Pulumi project on our org
  standard: ESC for config, OIDC for auth,
  preview-before-apply. Use when asked to
  start a new stack, service, or IaC project.
---

# Pulumi stack bootstrap

1. Ask cloud (aws|gcp|azure) + language (ts|py|go).
2. `pulumi new <cloud>-<language> --name <name>`
3. Wire config to ESC, not local secret files.
4. Always `pulumi preview` before `pulumi up`.
```

YAML frontmatter (`name` + `description`) on top, plain-Markdown instructions below. That's the whole format.

Drop that folder in one of a few places:

- `.claude/skills/<name>/` this project (commit to share)
- `~/.claude/skills/<name>/` you, across every project
- `<plugin>/skills/<name>/` from a plugin, namespaced
- Enterprise: managed settings, org-wide

Same name in two spots? Highest wins: **Enterprise** > **Personal** > **Project**

Frontmatter is a control panel

```yaml
---
name: deploy-service
description: Deploy a service to production
             # ← the trigger (default: Claude
             #    fires it AND you get /deploy-service)

disable-model-invocation: true
             # ← only YOU: /deploy-service
user-invocable: false
             # ← or only CLAUDE (pick one)

context: fork # ← runs in its own subagent,
agent: Explore #   own context, reports back
model: haiku  # ← cheap model for mechanical work

allowed-tools: Read Grep Bash(kubectl get:*)
             # ← least privilege, pre-approved
---
```

Who invokes it?

- **Default:** both. Claude auto-triggers on the description; every skill is also a `/slash-command`
- `disable-model-invocation: true` → **you-only.** Right for anything that mutates: deploys, rollbacks, `pulumi up`
- `user-invocable: false` → **Claude-only.** Background knowledge, hidden from the `/` menu
- `context: fork` → the skill runs in a **separate subagent**: doesn't fill your context, can use a different model, returns just the result

Anatomy of a *complex* skill

```
incident-triage/
├── SKILL.md              # the procedure, concise
├── references/
│   ├── severity-matrix.md
│   └── escalation.md
├── scripts/
│   └── gather-diagnostics.sh
└── templates/
    └── postmortem.md
```

- **SKILL.md** stays short. It's what loads on trigger.
- **references/** hold the long detail, read only when needed.
- **scripts/** do the deterministic steps, so the model never re-derives a fragile command.
- **templates/** are the artifacts it fills in (a postmortem, a PR body).

DEMO a real one — `service-connectivity-triage` , a PR against our team skills repo →

Skills come from work you already did

- Something broke. You and the agent dug through it. It's fixed.
- That session **is the runbook** — the commands that worked, the dead ends, the tell that cracked it
- Afterwards: reopen the session, extract the skill. Anthropic's `skill-creator` does the drafting
- PR it to the team repo → `apm install` ships it to **everyone's agent**

The best time to write a skill is right after you needed one.

`cc-pick`
browse + resume past sessions, in the terminal

`ccview`
web viewer for session history

`skill-creator`
Anthropic's skill-that-writes-skills — `anthropics/skills` (in our apm.yml)

Progressive disclosure: why you can have dozens

LEVEL 1 · ALWAYS LOADED

`name` + `description`

of *every* skill · ~100 tokens each

LEVEL 2 · ON TRIGGER

the `SKILL.md` body

loads only when the task matches

LEVEL 3 · ON DEMAND

scripts · references · templates

loads only when a step needs it

Install **dozens** of skills. Only the one the agent actually uses ever reaches the **context window**.

Why

DevOps work is skill-shaped

what · **why** · get · wire · use · build

The job is already skill-shaped

"If you find yourself doing the same type of task with different content each time, that's a skill waiting to be built."

Pipeline triage. Pulumi drift. `CrashLoopBackOff`. Cost spikes. The Sev1 at 2am. That's `repeatable judgement`. Not novel code.

Pulumi's own number: **LLMs now run over 20% of infrastructure deployments**, heading past 50% this year. The agent is already in your infra. The only question is whether it follows `your` playbook.

Turn the wiki nobody reads into something the agent runs

TODAY

- Runbook in a wiki, last edited 18 months ago
- Senior engineer re-explains it on every incident
- Agent guesses, reaches for the wrong command
- You re-paste your conventions into every prompt

WITH SKILLS

- Runbook is a `SKILL.md` in git, reviewed like code
- Encoded **once**, so every engineer's agent runs it the same way
- Safe by default: read-only, preview before apply
- Triggers itself when the task matches

The stack in 2026: skills carry the **know-how**, MCP servers / CLIs give **governed access** to real systems (Pulumi, Vault, Datadog), and the agent **orchestrates**.

Get

find & evaluate existing skills

what · why · **get** · wire · use · build

Don’t write what you can adopt

- **Vendor sets:** Pulumi ships `pulumi/agent-skills` (Apache-2.0, 4 plugin groups); cloud providers ship theirs
- The skills directory, `skills.sh`, and Anthropic's `anthropics/skills`
- Community runbooks like `bregman-arie/devops-sre-skills` and `dirien/claude-skills`
- And your own org repo, full of skills your teammates already wrote

```
# The fastest skill is the one
# someone else already ran in prod.

npx skills find pulumi      # search
npx skills find kubernetes
```

Evaluate before you trust

A skill **acts on your infrastructure.**

- **Who publishes it?** Unknown author = unknown blast radius.
- **What can it run?** Read the scripts.
- **Is the description honest?** It's the trigger.
- **Safe by default?** Preview before mutate.
- **Maintained?** Commits, versions, changelog.
- **Fits your stack?** Pulumi ≠ Terraform.

> **RULE OF THUMB**
>
> Read the whole `SKILL.md` and every script it bundles before you enable it. **If you wouldn't merge it as a PR, don't install it as a skill.**

Wire

one manifest for the whole team — APM

what · why · get · **wire** · use · build

Connecting a skill: two ways

UNIVERSAL CLI (SKILLS.SH)

```
# Works for Claude Code, Cursor,
# Copilot, Codex, Gemini, …
npx skills add pulumi/agent-skills \
  --skill '*'
```

Lands in `.agents/skills/`, pinned in `skills-lock.json`.

CLAUDE CODE MARKETPLACE

```
/plugin marketplace add \
  pulumi/agent-skills
/plugin install pulumi
```

Lands in your Claude plugins, grouped by plugin.

Both are per-machine. What does your **teammate** get when they clone the repo?

Nothing. Unless there's a manifest.

APM: package.json for your agent

The course uses `npx skills`. Perfect for one laptop. **Microsoft APM** takes the next step: a `manifest` your whole team shares.

Declare skills, MCP servers, and LSP servers in `apm.yml`; drop hooks and your own skills under `.apm/`. Run `apm install`. Every teammate's agent (Claude, Copilot, Cursor, Codex, Gemini) gets the **same setup**.

Reproducible by design. And because these skills run code, `apm` scans every install for `hidden-Unicode attacks` and pins their hashes. Policy can gate what's allowed across the org.

DEMO run it — `apm install`, then what landed in `.claude/`

github.com/microsoft/apm

One manifest → every agent

```yaml
# apm.yml: committed to the repo, the same for everyone
name: getting-started-with-devops-ai-skills
targets: [claude]            # also: copilot, cursor, codex, gemini, …
dependencies:
  apm:
    - pulumi/agent-skills/pulumi          # Pulumi's core skill set
    - pulumi/agent-skills/delegation      # pulumi-neo-handoff
    - bregman-arie/devops-sre-skills/skills/kubernetes/diagnose-crashloop
  mcp:
    - name: pulumi                        # hosted MCP server (OAuth on first use)
      transport: http
      url: https://mcp.ai.pulumi.com/mcp
  # lsp: language servers (next slide) · hooks live in .apm/hooks/
```

APM INSTALL

Resolve + scan, then integrate it all into `.claude/` : skills, hooks, LSP, MCP.

COMMIT APM.LOCK.YAML

Pins exact versions and content hashes, the same for everyone.

GIT CLONE && APM INSTALL

A new teammate is fully configured in one command.

Skills, MCP, Neo: three layers

AGENT SKILLS

Static know-how in your agent. "How an expert uses Pulumi." Loaded on demand. Free until used.

PULUMI MCP SERVER

Live tools. Query the registry, validate code, reach Pulumi Cloud. Governed access, so the agent never holds creds.

PULUMI NEO

Autonomous agent with RBAC + human-in-the-loop. Hand off with the `pulumi-neo-handoff` skill.

A **skill** teaches the agent how to drive the **MCP** server; when the job gets big, `pulumi-neo-handoff` packages it into a **Neo** task.

Quality depends on how you configure it

That stack is only as good as its configuration. A coding agent can write Pulumi for you, but **how well depends almost entirely on how you've set it up.**

Skills and the MCP server are two levers. Two more keep it from drifting off your patterns:

LSP lets it see real types as it writes. **Hooks** and **permissions** are local guardrails on the dangerous commands. `apm install` wires the hook; you add a permissions deny-list.

LSP: let the agent see real types

```yaml
# apm.yml: a language server per Pulumi language
dependencies:
  lsp:
    - name: typescript-language-server
      command: typescript-language-server
      args: ["--stdio"]
      extensionToLanguage: { ".ts": typescript }
    - name: pyright                          # Python
      command: pyright-langserver
      args: ["--stdio"]
      extensionToLanguage: { ".py": python }
    - name: gopls                            # Go
      command: gopls
      args: ["serve"]
      extensionToLanguage: { ".go": go }
```

- The agent sees **real types and live diagnostics** as it writes
- So it uses the `real cloud-SDK API` (the actual args on `aws.s3.Bucket`) instead of guessing
- `apm install` writes `.lsp.json`, not the binaries. You put `gopls` / `pyright` / `typescript-language-server` on `$PATH` yourself
- **Mistakes get caught as the code is typed, not at `pulumi up`**

Hooks: a fast local guardrail (not a wall)

`.APM/HOOKS/BLOCK-PULUMI-MUTATIONS.JSON`

```
{ "PreToolUse": [{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
    "command": "\"$CLAUDE_PROJECT_DIR\"/scripts/guard-pulu
    "timeout": 10
  }]
}]}
```

```
# guard-pulumi.sh blocks (exit 2) pulumi up/update/
# destroy, flags and all; fails CLOSED if it can't parse.
```

- A `PreToolUse` hook on the **Bash** tool sees the command first; exit `2` blocks it and hands the reason back
- `apm install` merges it into `.claude/settings.json`. Pair it with a `permissions` deny-list, the "settings" lever
- **It's a nudge, not a wall. The agent can still deploy via the Automation API, the MCP server, or a `make` wrapper, none of which touch Bash**

SO WHERE'S THE REAL WALL?

Server-side: **Pulumi Cloud deployment policies + OIDC-scoped, short-lived creds.** The hook catches the obvious local mistake fast; the Cloud policy is what actually stops a bad apply.

Use

the configured agent, at work

what · why · get · wire · **use** · build

Same prompt, configured vs not

NAKED AGENT

```
// "make an S3 bucket for our data"
new aws.s3.BucketV2("data", {
  publicReadAccess: true,  // ❌ invented arg
});
// then runs `pulumi up`, no preview
```

Hallucinated API, no tags, deploys unprompted.

CONFIGURED AGENT

```
// same prompt; skills + LSP + hooks on
new aws.s3.BucketV2("data", {
  tags: { team: "platform", env: "dev" },
});
// `pulumi preview` ✅ (hook held `up`)
```

LSP killed the bad arg; skill added tags; it stopped at preview.

DEMO for real — the squiggle, the skill firing itself, the hook holding `pulumi up`

Use it: the agent triggers the skill itself

You type a normal request:

> "Start a new **payments** service, AWS, TypeScript."

- Agent matches it to `golden-path-service` from its **description**
- Loads the body, follows the steps, runs the bundled script
- `previews`, waits for your OK, and never runs `pulumi up` unprompted

```
🛠️  golden-path-service triggered
→ pulumi new aws-typescript …
→ wired config to ESC (OIDC, no keys)
→ applied standard tags
→ pulumi preview  ✅  (awaiting approval)
```

DEMO …and now that `payments` service is crashlooping — `incident-triage`, live

Build

your own skills, done well

what · why · get · wire · use · **build**

Six principles that make a skill reliable

1. The description is the trigger. Say what it does *and* when. Third person, specific.

2. One skill, one job. Sharp scope beats a mega-skill that competes with itself.

3. Keep `SKILL.md` short. Under ~500 lines; push detail into `references/`.

4. Scripts for procedure. If a step is exact commands, ship the script and have the agent run it.

5. Name it like your team talks. Match the words people actually use.

6. Safe by default. Read-only, preview, confirm before mutating. No static creds.

> **IF YOU REMEMBER ONE THING**
>
> The `description` is the most important line you write. It's how the agent decides to load the skill at all.

Keep them current · remove them clean

KEEP UP TO DATE

- Skills live in **git** next to the runbook; change both in one PR
- **Review like code**; pin deps and bump deliberately (`apm update`)
- Re-test on model/agent upgrades, and keep a smoke prompt per critical skill
- **Stale skill > stale doc: the agent *acts* on it**

REMOVE CLEANLY

- Every skill costs context and a chance to mis-trigger, so prune the unused
- `apm uninstall <pkg>` , or delete `.apm/skills/<name>/` and recompile
- Drop the manifest entry so it doesn't come back on the next sync
- Unsure? **Disable** beats delete (`skillOverrides`)

One loop, closed today

- An **incident**: checkout dark, every pod green — nothing any installed skill covered
- The debugging **session became a skill**: `skill-creator` extracted `service-connectivity-triage`
- The skill became **PR #1** on our team repo — reviewed like code
- Merged → one line in `apm.yml` → next `apm install` : it's in **every teammate's agent**, pinned

incident → session → skill → PR → review → team.

Your runbooks are skills waiting to be built — capture the next one you do twice.

What we didn't cover

SKILL EVALS

Anthropic's `skill-creator` now **tests skills**: eval cases (a realistic prompt + assertions), benchmarks for pass rate / time / tokens, and blind A/B of skill vs no skill.

ORG-WIDE GOVERNANCE

Enterprise-managed skills, and APM's `apm-policy.yml`: tighten-only rules from enterprise → org → repo for what agents may install.

And the habit that makes evals matter: **every new model release can change how your skills behave.** Re-run the evals; a skill that worked yesterday can drift today.

You used the whole Pulumi stack today

Agent Skills	The skills `apm install` wired in — `pulumi/agent-skills`, open to your contributions
MCP server	Registry, validation, Pulumi Cloud — governed live access; the agent never holds creds
ESC	Where golden-path put the config — OIDC, no static secrets anywhere the agent can reach
Cloud policy	The real wall behind the hook — deployment policies + short-lived scoped creds
Neo	Hand off whole jobs — RBAC + human approval. **And Neo runs Agent Skills:** the skill you watched get born teaches it too

Same skills, every agent — including the autonomous one. pulumi.com/docs/ai · app.pulumi.com/signup

THANK YOU

Go make your agent yours.

Engin Diri
Pulumi
dirien · engin-diri

Adam Gordon Bell
Pulumi
adamgordonbell · adamgordonbell

Workshop repo
slides · chapters · skills

Pulumi Skills
pulumi.com/docs/ai